

4th May 2026

# second assignment

## Team

Martim Santos, 114614  
Sebastião Teixeira, 114624  
Duarte Santos, 113304  
Henrique Teixeira, 114588

## Subject

AS







## Scenario C

### Lost Sales

WMS unavailability must not block order acceptance. Orders queue locally and sync when the system recovers.

### Overselling

Stock changes in physical stores must reflect on the website within seconds to prevent overselling.

### Operational Blindness

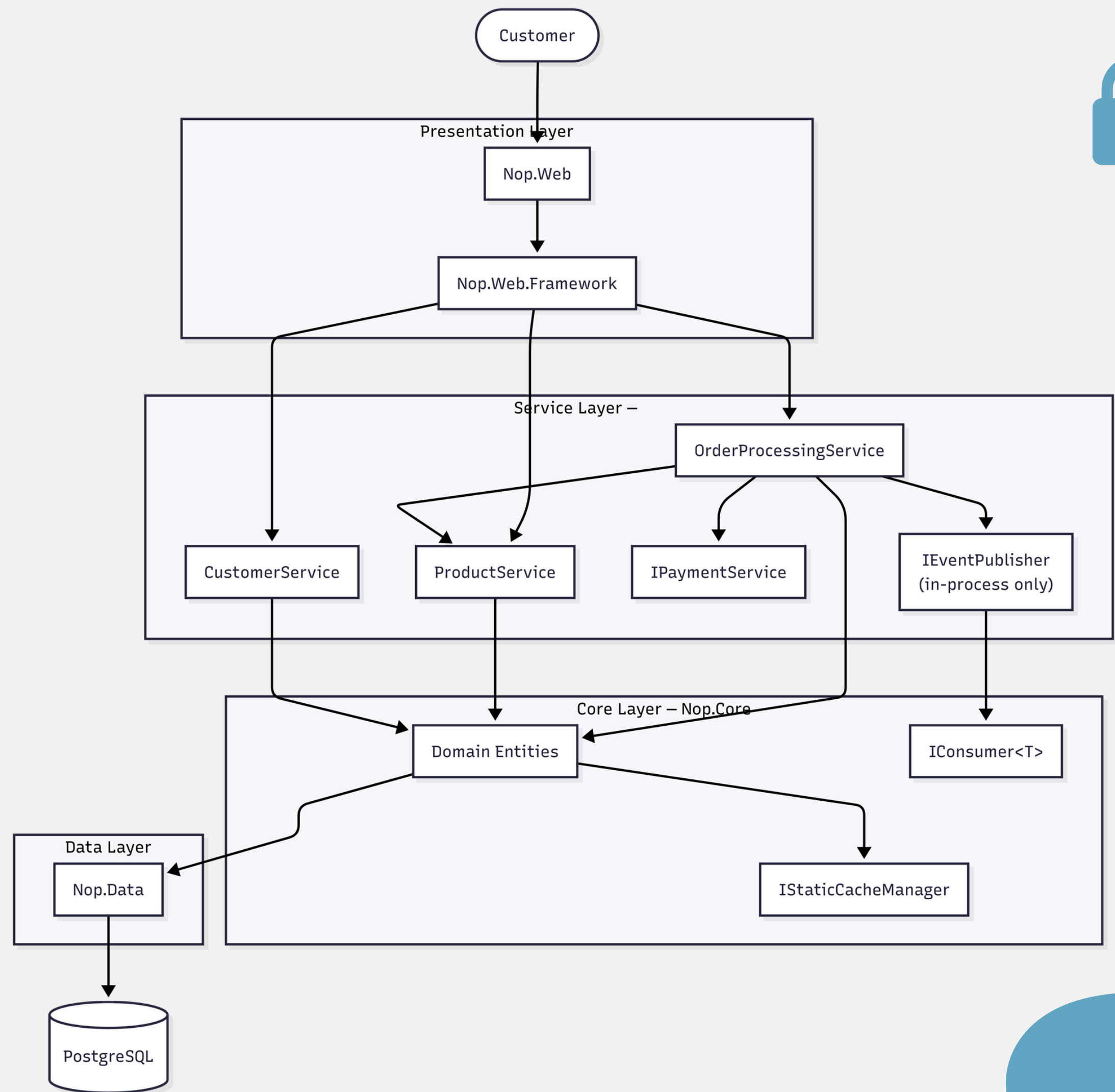
Every online order must reach ERP and WMS reliably, with full traceability across system boundaries.

# Current State



## Architecture

- Modular monolith – ASP.NET Core 10 + PostgreSQL
- Single deployable unit: Nop.Web
- In-process IEventPublisher – no cross-process events
- No outbox, no broker, no external integrations



# Bounded Contexts



## noCommerce Monolith

- **Identity (cross-cutting)**
- **Order Management (Core):**
  - Orders, items, outbox, payment status
- **Catalog & Pricing:**
  - Products, stock quantities, pricing

## Extracted Services

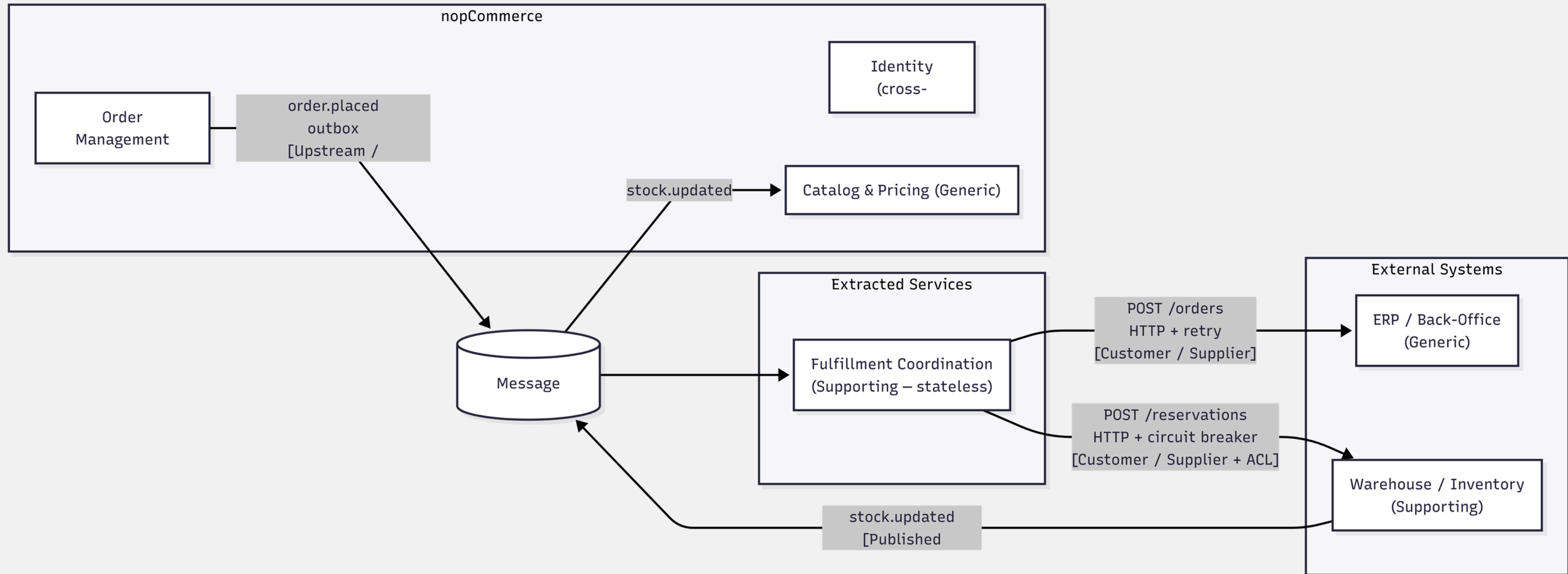
- **Fulfillment Coordination:**
  - Stateless — no DB of its own
  - Consumes order.placed from RabbitMQ
  - Forwards to ERP (retry) + WMS (circuit breaker)

## External Systems

- ERP / Back-Office (Generic subdomain)
  - In-memory stub — HTTP + retry
- Warehouse / Inventory (Supporting)
  - In-memory stub — the pressure point
  - Publishes stock.updated back to RabbitMQ

**No shared database. Communication is solely via events (RabbitMQ) or HTTP with well-defined contracts.**

# Bounded Contexts



# Quality Attribute



|      | Attribute      | Stimulus                  | Measurable Response                                                   |
|------|----------------|---------------------------|-----------------------------------------------------------------------|
| QA-1 | Availability   | WMS unavailable           | 0% orders blocked; events queued → delivered in < 60 s after recovery |
| QA-2 | Consistency    | Physical store sale       | Stock update in nopCommerce web store within 30 s of WMS event        |
| QA-3 | Recoverability | WMS recovers after outage | All DLQ messages processed within 60 s; zero duplicate reservations   |
| QA-4 | Observability  | WMS becomes slow          | Dashboard shows OPEN state within 5 s of first WMS timeout            |
| QA-5 | Reliability    | ERP transient 503 *2      | Order confirmed in ERP in < 30 s via exponential backoff retry        |



# Chosen Framework



Our Choice

ADM

Designed for large  
organizations transforming  
it landscapes

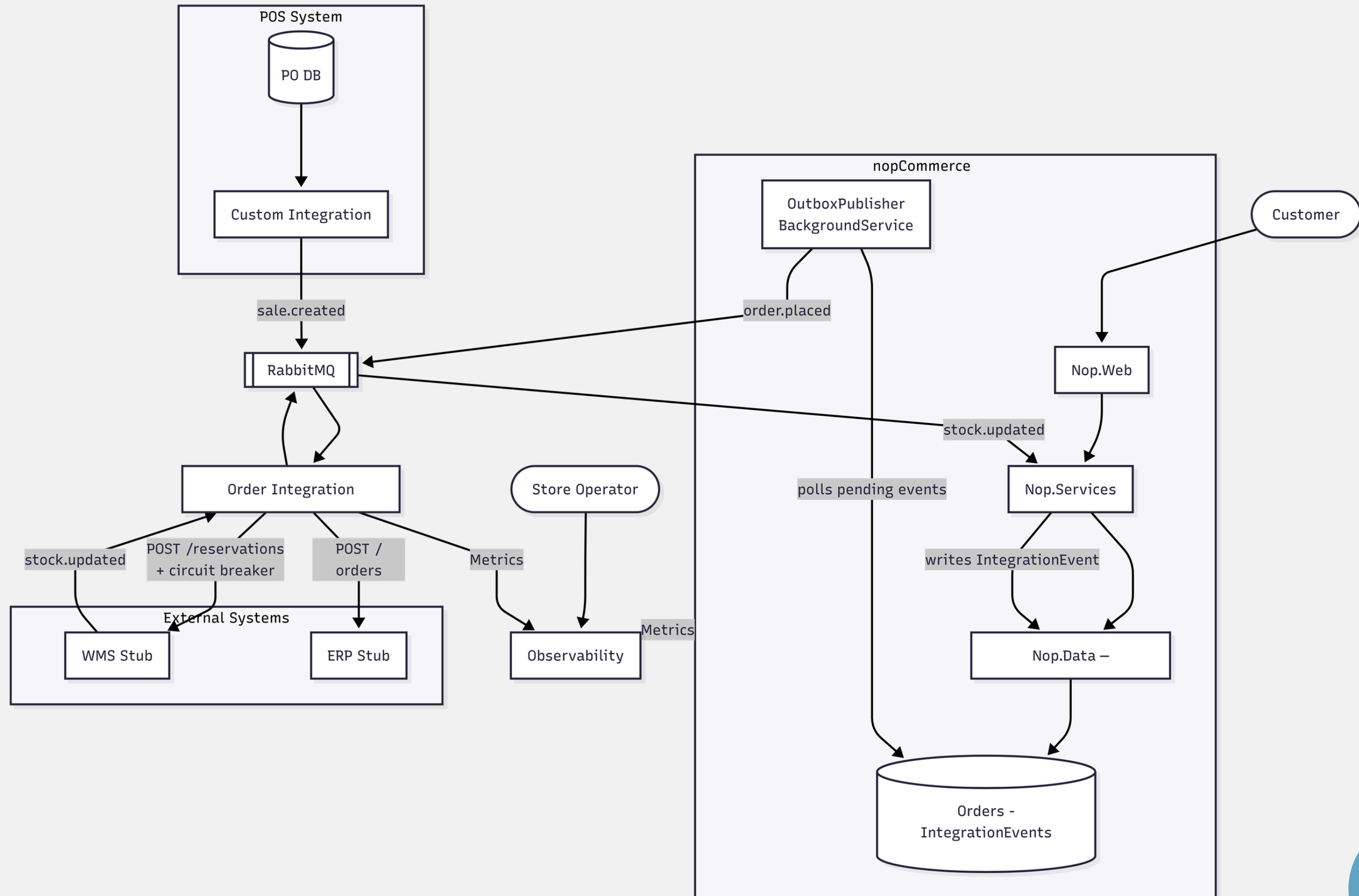
ACDM

We have one clear  
scenario with defined use  
cases

ADD

Every structural decision  
traces to a quality  
requirement.

# Target Architecture





# ADRs



|        |                                                    |                                                                                                                                                                                                                                                     |
|--------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ADR-01 | <b>RabbitMQ over Kafka</b>                         | Native Dead-Letter Exchange (DLX) required for WMS DLQ pattern. Kafka needs custom offset management — no native DLX. Throughput needs < 1 000 msg/s. RabbitMQ adds zero ops overhead.                                                              |
| ADR-02 | <b>Outbox Pattern over Direct Publish</b>          | Direct publish inside PlaceOrderAsync creates dual-write gap: order saved but event silently lost if broker is down at commit time. Outbox write is atomic with order — event guaranteed even on crash.                                             |
| ADR-03 | <b>.NET Worker Service for Integration Service</b> | Polly circuit breaker and retry are first-class in .NET. Python tenacity lacks a clean circuit breaker abstraction.                                                                                                                                 |
| ADR-04 | <b>WMS + ERP Stubs over Real Systems</b>           | Assignment brief explicitly permits surrogate simulators. Real OpenBoxes = 2 GB image + Java + no native failure injection. Stubs provide identical architectural pressure with full failure control. For POS we will se Open Source Point of Sale. |

# Risk Plan



|                                                   | Severity | Problem                                                              | Risk                                       | Mitigation                                                                                                                                                                                                         |
|---------------------------------------------------|----------|----------------------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Outbox Pattern Integration</b>                 | HIGH     | nopCommerce internal modification (DI, migrations, background tasks) | Events never publish if integration fails  | <ul style="list-style-type: none"><li>• Spike feasibility test [solved]</li></ul>                                                                                                                                  |
| <b>Cross-Channel Inventory Conflicts</b>          | MEDIUM   | POS + Web simultaneous orders → race condition → overselling         | Violates stock accuracy requirement (QA-2) | <ul style="list-style-type: none"><li>• Event timestamp ordering for automatic resolution</li><li>• Prefer cancel online over physical sell</li><li>• Time window tolerance (30 seconds)</li></ul>                 |
| <b>Event Idempotency and Duplicate Processing</b> | MEDIUM   | Event Idempotency Failures                                           | Violates inventory correctness             | <ul style="list-style-type: none"><li>• eventId (UUID) as deduplication key</li><li>• Integration Service tracks processed eventIds (in-memory cache)</li><li>• WMS stub ignores duplicate reservationId</li></ul> |

# Evolution Roadmap



## Phase 0 - Baseline

- Standalone nopCommerce + PostgreSQL;
- No external integrations;
- Problem study and solution design

## Phase 1 - Outbox + Message Backbone

- IntegrationEvent table (FluentMigrator)
- Outbox Publisher Background Service

## Phase 2 - Fulfillment Happy Path

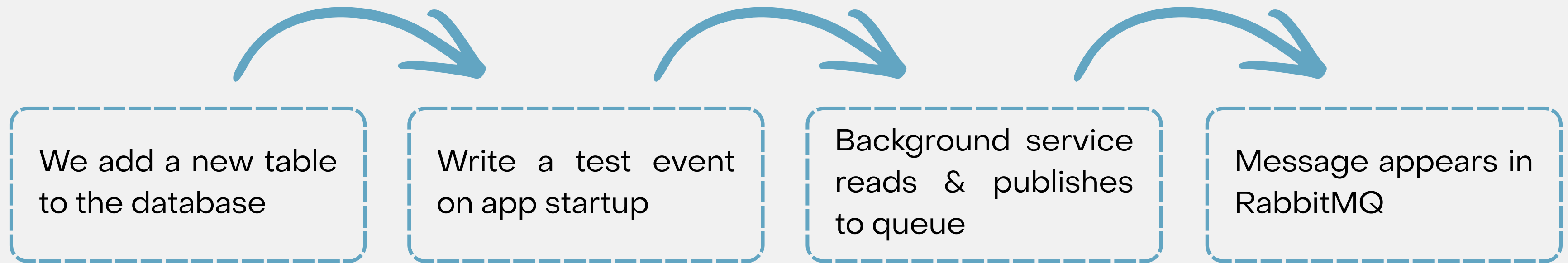
- Order Integration Service;
- ERP Stub + WMS Stub;
- StockUpdateConsumerB ackgroundService;
- Point of Sale Integration Service

## Phase 3 - Resilience + Pressure Point

- Circuit breaker + DLQ + Reconciliation loop;
- Observability Dashboard (live state);



# Feasibility Spike Test



## Success Criteria

Message appears in RabbitMQ within 10s of startup;  
Row marked as Published = true in database;  
No DI registration errors or crashes.

**any questions?**